

EXPERIMENT – 2 (Question 1)

Name: K ANSHU

Roll No: 252003044

Section: 8

Title

Installation of Linux Development Environment, Git Repository Setup, Project Structure Creation, Process Abstraction using fork(), Process Tree Analysis, and System Call Tracking.

Aim

To install and verify the Linux development environment, configure GCC and Git, create a project repository and directory structure, build an initial Makefile, implement process creation using the fork() system call, inspect the process hierarchy, and analyze system calls using strace.

Theory / Algorithm / Code

The experiment demonstrates the Linux development workflow by configuring GCC and Git, creating a project structure, compiling a C program using a Makefile, and understanding process creation through the fork() system call. The parent and child processes are identified using their Process IDs (PIDs). The Linux process hierarchy is observed using pstree, while strace is used to monitor system calls during execution.

Procedure

Step 1: Verify GCC Installation

```
gcc --version
```

[GCC Version]

```
k_anshu@DESKTOP-TM20BAQ:~/OS_Week2$ gcc --version
gcc (Ubuntu 15.2.0-16ubuntu1) 15.2.0
Copyright (C) 2025 Free Software Foundation, Inc.
This is free software; see the source for copying conditions. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.
```

Step 2: Verify and Configure Git

```
git --version
```

```
git config --global user.name "K ANSHU"
```

```
git config --global user.email "kanshu@klh.edu.in"
```

```
git config --list
```

[Git Configuration]

```
k_anshu@DESKTOP-TM20BAQ:~/OS_Week2$ git --version
git version 2.53.0
k_anshu@DESKTOP-TM20BAQ:~/OS_Week2$ git config --list
user.name=K ANSHU
user.email=kanshu@klh.edu.in
core.repositoryformatversion=0
core.filemode=true
core.bare=false
core.logallrefupdates=true
```

Step 3: Create Project Directory

```
mkdir OS_Week2
cd OS_Week2
pwd
```

[Project Directory]

```
k_anshu@DESKTOP-TM20BAQ:~/OS_Week2$ pwd
/home/k_anshu/OS_Week2
```

Step 4: Create Project Structure

```
mkdir src
mkdir bin
mkdir docs

touch src/fork_demo.c
touch Makefile
touch README.md

tree
```

[Project Structure]

```
k_anshu@DESKTOP-TM20BAQ:~/OS_Week2$ tree
```

```
.
├── Makefile
├── README.md
├── bin
│   └── fork_demo
├── docs
├── src
│   └── fork_demo.c
```

```
4 directories, 4 files
```

C Program

```
#include <stdio.h>
#include <unistd.h>
```

```
int main()
{
    pid_t pid = fork();

    if(pid == 0)
    {
        printf("Child Process\n");
        printf("Child PID = %d\n", getpid());
    }
    else
    {
        printf("Parent Process\n");
        printf("Parent PID = %d\n", getpid());
    }

    return 0;
}
```

[fork_demo.c]

```
k_anshu@DESKTOP-TM20BAQ:~/OS_Week2$ cat src/fork_demo.c
#include <stdio.h>
#include <unistd.h>

int main()
{
    pid_t pid = fork();
    if (pid == 0)
    {
        printf("Child Process\n");
        printf("Child PID = %d\n", getpid());
    }
    else
    {
        printf("Parent Process\n");
        printf("Parent PID = %d\n", getpid());
    }
    return 0;
}
```

Makefile

```
all:
    gcc src/fork_demo.c -o bin/fork_demo

clean:
    rm -f bin/fork_demo
```

[Makefile]

```
k_anshu@DESKTOP-TM20BAQ:~/OS_Week2$ make
gcc src/fork_demo.c -o bin/fork_demo
```

Compilation

Command: make

Output: gcc src/fork_demo.c -o bin/fork_demo

[Compilation Output]

```
k_anshu@DESKTOP-TM20BAQ:~/OS_Week2$ ./bin/fork_demo
```

Program Execution

Command: ./bin/fork_demo

Sample Output:

Parent Process

Parent PID = 9905

Child Process

Child PID = 9906

```
k_anshu@DESKTOP-TM20BAQ:~/C$ _Week2$ !./bin/fork_demo
Parent Process
Parent PID = 9905
Child Process
Child PID = 9906
```

[Process Tree Output]

Important calls: `execve()`, `clone()`, `getpid()`, `write()`, `exit_group()`.

[strace Output]

```
k_anshu@DESKTOP-TM20BAQ:~/OS_Week2$ vi fork_demo.c
#include <sys/types.h> /* for pid_t */
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
#include <sys/wait.h>
#define NULL 0
int main()
{
    pid_t pid;
    int status;
    printf("Parent Process\n");
    pid = fork();
    if (pid < 0) {
        printf("Fork Failed\n");
        exit(1);
    }
    if (pid > 0) {
        /* Parent process */
        printf("Parent PID = %d\n", getpid());
        printf("Child PID = %d\n", pid);
        /* Parent waits for child to terminate */
        wait(&status);
        printf("Parent Process\n");
        printf("Parent PID = %d\n", getpid());
        printf("Child PID = %d\n", pid);
    }
    else {
        /* Child process */
        printf("Child Process\n");
        printf("Child PID = %d\n", getpid());
        /* Child exits */
        exit(0);
    }
    return 0;
}
```

Git Commit

git add .

git commit -m "Week 2 OS Experiment 1 completed"

[Git Commit / git log --oneline]

```
k_anshu@DESKTOP-TM20BAQ:~/OS_Week2$ git log --oneline
a7a972c (HEAD -> master) Week 2 OS Experiment 1 completed
```

Result

The Linux development environment was configured successfully. The project structure was created, the program using fork() was compiled and executed successfully, the process tree was inspected, system calls were analyzed using strace, and the work was committed to the local Git repository.